

Ministry of Education and Science of Ukraine
National Technical University of Ukraine
"Kyiv Polytechnic Institute named after Igor Sikorsky"
Educational and Scientific Institute of Atomic and Thermal Energy
Department of Digital Technologies in Energy

Graphics work

Topic: «Operations on texture coordinates.»
from the discipline: «Visualization of graphical and geometric information»

Variant 10

Performed by:
student of the TP-52mp group
Pidherskyi I.O.

Kyiv – 2025

1 TASK

This calculation and graphic work focuses on texture coordinate transformations in WebGL applications. The task extends the Control Task (PA3) implementation with dynamic texture manipulation capabilities. The core objective is implementing real-time UV coordinate rotation around a user-controlled center point.

Variant 10 falls under the “even variants” category, which requires texture rotation implementation (as opposed to texture scaling for odd variants). The rotation operation transforms texture coordinates in the UV space, creating a visual effect where the texture pattern rotates around a specified anchor point on the surface.

The implementation reuses the texture mapping infrastructure from PA3, including three texture types: diffuse map for base color, specular map for highlight intensity, and normal map for surface detail. All three textures must rotate synchronously around the same center point to maintain visual consistency.

The center point position is controlled via keyboard input. Keys A and D adjust the U coordinate (horizontal axis in texture space), while keys W and S adjust the V coordinate (vertical axis). Both coordinates are constrained to the normalized range $[0, 1]$, representing the full texture extent. This constraint prevents invalid texture lookups and ensures the rotation center always lies within the texture bounds.

The parametric surface used is the Surface of Conjugation of Two Coaxial Cylinders. This surface smoothly transitions between two cylinders of different radii ($R1 = 0.5$ and $R2 = 1.5$) over a height parameter. The cylindrical topology provides a natural test case for texture rotation, as the texture wraps around the circumference and rotation effects are clearly visible.

The deliverables include a functional WebGL application with interactive controls for adjusting rotation parameters, and this technical report documenting the implementation approach, theoretical foundations, and usage instructions. The application runs in any modern web browser without requiring plugins or additional software installation.

2 THEORY

2.1 WebGL Rendering Pipeline

WebGL (Web Graphics Library) provides a JavaScript API for rendering 2D and 3D graphics in web browsers without plugins. It is based on OpenGL ES 2.0 and executes on the GPU for hardware-accelerated rendering.

The rendering pipeline processes geometry through several stages. Vertex data stored in Vertex Buffer Objects (VBOs) enters the vertex shader, which transforms each vertex position from model space to clip space. The GPU then assembles vertices into primitives (triangles) and performs rasterization — converting vector geometry into fragments (pixel candidates). The fragment shader computes the final color for each fragment, which is then written to the framebuffer for display.

Shader programs are written in GLSL (OpenGL Shading Language), a C-like language compiled at runtime. Uniforms pass data from JavaScript to shaders (matrices, textures, parameters), while attributes provide per-vertex data (positions, normals, texture coordinates).

2.2 Texture Mapping Fundamentals

Texture mapping assigns 2D image data to 3D surface geometry. Each vertex carries UV coordinates (also called texture coordinates) specifying which texel (texture pixel) maps to that location. The UV space is normalized: (0, 0) represents the bottom-left corner and (1, 1) the top-right corner of the texture.

During rasterization, UV coordinates are interpolated across triangle surfaces. The fragment shader samples the texture at interpolated UV positions using the `texture2D()` function. WebGL supports multiple texture units, allowing simultaneous access to several textures — in this implementation, texture unit 0 holds the diffuse map, unit 1 the specular map, and unit 2 the normal map.

Texture wrapping modes control behavior when UV coordinates exceed [0, 1]. REPEAT mode tiles the texture, CLAMP_TO_EDGE pins edge texels, and MIRRORED_REPEAT creates reflections. Filtering modes (NEAREST, LINEAR, mipmapping) control interpolation quality.

2.3 2D Rotation Transformation

Rotation in 2D is an affine transformation that moves points around a fixed center. The standard rotation formula for angle θ around the origin is:

$$x' = x * \cos(\theta) - y * \sin(\theta)$$

$$y' = x * \sin(\theta) + y * \cos(\theta)$$

This can be expressed in matrix form as:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

To rotate around an arbitrary point (cx, cy) rather than the origin, the transformation becomes translate-rotate-translate:

1. Translate point to origin: (x - cx, y - cy)
2. Apply rotation around origin
3. Translate back: add (cx, cy)

The combined formula:

$$x' = (x - cx) * \cos(\theta) - (y - cy) * \sin(\theta) + cx$$

$$y' = (x - cx) * \sin(\theta) + (y - cy) * \cos(\theta) + cy$$

2.4 Rotation in Texture Space

Applying 2D rotation to texture coordinates produces UV rotation. When implemented in the vertex shader, rotation happens before texture sampling — the shader outputs transformed UV coordinates that the fragment shader uses directly.

Vertex shader implementation is preferable to fragment shader because UV coordinates are per-vertex attributes interpolated by the GPU. Computing rotation per-vertex (typically thousands of vertices) is more efficient than per-fragment (potentially millions of pixels).

The rotation center in UV space determines the visual pivot point on the textured surface. Center (0.5, 0.5) rotates around the texture middle; (0, 0) rotates around the bottom-left corner, causing the texture to “swing” around that point.

2.5 Normal Mapping Integration

Normal mapping uses a texture to store surface normal perturbations, adding geometric detail without additional mesh complexity. The normal map encodes tangent-space normals as RGB values, where (128, 128, 255) represents an unperturbed normal.

For correct lighting after UV rotation, the TBN (Tangent-Bitangent-Normal) matrix transforms sampled normals from tangent space to eye space. The implementation computes per-vertex tangents from the surface parameterization and constructs TBN in the fragment shader using Gram-Schmidt orthogonalization.

When texture coordinates rotate, the sampled normal direction rotates with them, maintaining consistent lighting across the surface.

3 IMPLEMENTATION

3.1 Application Architecture

The application consists of six source files with clear separation of concerns:

File	Responsibility
index.html	Page structure, UI controls, CSS styling
main.js	WebGL initialization, render loop, event handlers
shader.gpu	GLSL vertex and fragment shaders
Model.js	Surface mesh generation, vertex buffer management
TextureHandler.js	Async texture loading with callbacks
Utils/	External libraries (trackball, matrix math)

3.2 Rotation State Management

Three global variables track the current rotation configuration:

```
let rot_center_u = 0.5; // U coordinate of rotation center
let rot_center_v = 0.5; // V coordinate of rotation center
let rot_angle_deg = 0;  // Rotation angle in degrees
```

The center initializes at (0.5, 0.5) — texture midpoint. The angle starts at 0° (no rotation). These values update via UI controls and keyboard input.

3.3 GLSL Rotation Function

The vertex shader defines the UV rotation function:

```
uniform vec2 u_rot_center;
uniform float u_rot_angle;

vec2 rotate_uv(vec2 uv, vec2 center, float angle) {
    vec2 d = uv - center;
```

```

float c = cos(angle);
float s = sin(angle);
return vec2(d.x * c - d.y * s, d.x * s + d.y * c) + center;
}

```

The function subtracts the center (translate to origin), applies rotation matrix components, and adds the center back. Trigonometric functions compute once per vertex.

The fragment shader samples all three textures at the rotated coordinates:

```

vec3 diffuse_color = texture2D(u_tex_diffuse, v_texcoord).rgb;
float specular_intensity = 1.0 - texture2D(u_tex_specular, v_texcoord).r;
vec3 tangent_normal = texture2D(u_tex_normal, v_texcoord).rgb * 2.0 - 1.0;

```

Phong lighting combines ambient, diffuse, and specular terms using the perturbed normal from the normal map.

3.4 Input Processing

The keyboard handler adjusts center coordinates:

```

function handle_key(e) {
  const step = 0.025;
  const key = e.key.toLowerCase();
  switch (key) {
    case 'a': rot_center_u = Math.max(0, rot_center_u - step); break;
    case 'd': rot_center_u = Math.min(1, rot_center_u + step); break;
    case 'w': rot_center_v = Math.min(1, rot_center_v + step); break;
    case 's': rot_center_v = Math.max(0, rot_center_v - step); break;
    default: return;
  }
  update_center_display();
}

```

The step size (0.025) provides fine control — 40 key presses traverse the full [0, 1] range. Math.max/min clamp values to valid UV bounds.

3.5 Uniform Updates

Each frame, the draw function passes rotation parameters to the shader:

```
gl.uniform2fv(sh_program.i_rot_center, [rot_center_u, rot_center_v]);  
gl.uniform1f(sh_program.i_rot_angle, rot_angle_deg * Math.PI / 180);
```

The angle converts from degrees (UI-friendly) to radians (shader-required) during transfer.

3.6 Surface Mesh Generation

The surface uses the formula from the Encyclopedia of Analytical Surfaces:

```
function calc_radius(alpha) {  
  const k = Math.PI / (2 * B);  
  return (R2 - R1) / 2 * (1 - Math.cos(k * alpha)) + R1;  
}
```

The mesh generator creates vertices with positions, normals, tangents, and UV coordinates. Data packs into an interleaved buffer (11 floats per vertex) for efficient GPU transfer.

4 USER INSTRUCTION

4.1 System Requirements

The application runs in any modern web browser with WebGL 1.0 support (Chrome, Firefox, Edge, Safari). No installation required — open the HTML file directly or through a local development server.

4.2 Starting the Application

For local development, use VS Code with the Live Server extension:

1. Open the project folder in VS Code
2. Right-click index.html in the file explorer
3. Select “Open with Live Server”
4. The browser opens automatically with the application

Alternatively, any HTTP server works (Python’s http.server, Node’s http-server, etc.). Opening index.html directly from the filesystem may cause texture loading issues due to browser security restrictions.

4.3 Application Interface

The interface consists of two main areas:

Left panel: WebGL canvas (600×600 pixels) displaying the textured 3D surface. The surface renders with Phong shading, three texture maps, and an orbiting point light.

Right panel: Control area with sliders and displays:

- Rotation Angle slider (0° – 360°)
- Center coordinates display (U and V values in green boxes)

- Keyboard hint showing W/A/S/D mappings
- Surface granularity sliders (U and V segments)



Fig 4.1. Application interface overview

4.4 View Manipulation

Rotating the view: Click and drag on the canvas to rotate the camera around the surface. The trackball controller allows free rotation in any direction.

Mesh resolution: Adjust the U Segments and V Segments sliders to change surface smoothness. Lower values (4–8) show the triangular mesh structure; higher values (50–72) produce smooth shading.



Fig 4.2. Surface at different mesh resolutions (less segments)

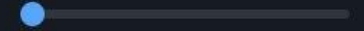
Surface of Conjugation of Two Coaxial Cylinders

Variant #10 | Texture Rotation



TEXTURE ROTATION

Rotation Angle 0°



Rotation Center:

U: 0.50 V: 0.50

W

V+

S

V-

A

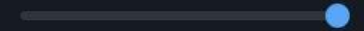
U-

D

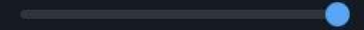
U+

SURFACE GRANULARITY

U Segments (height) 50



V Segments (angle) 72



Shading: Per-pixel Phong

Normal map: TBN matrix

Light: Orbiting point

Fig 4.3. Surface at different mesh resolutions (more segments)

4.5 Texture Rotation Controls

Setting rotation angle: Use the Rotation Angle slider to set the texture rotation from 0° to 360°. The texture rotates in real-time as you adjust the slider.

Moving the rotation center: Press keyboard keys to shift the pivot point:

- **W** — Move center up (V increases by 0.025)
- **S** — Move center down (V decreases by 0.025)

- **A** — Move center left (U decreases by 0.025)
- **D** — Move center right (U increases by 0.025)

The current center coordinates display in the green boxes. Both values clamp to [0, 1].



Fig 4.4. Texture rotation at 90° with center at (0.5, 0.5)



Fig 4.5. Same rotation angle with center moved to (0.25, 0.75)

4.6 Observing the Effect

To understand the rotation behavior:

1. Set angle to 45° with center at (0.5, 0.5) — texture rotates around its middle
2. Move center to (0, 0) with A and S keys — rotation pivots at the corner
3. Move center to (1, 1) — pivot shifts to opposite corner
4. Compare how the texture pattern shifts differently at each pivot location

The normal mapping continues to work correctly — lighting responds to the rotated surface detail texture.

5 SOURCE CODE

5.1 Vertex Shader (shader.gpu)

```
attribute vec3 a_vertex;
attribute vec3 a_normal;
attribute vec3 a_tangent;
attribute vec2 a_texcoord;

uniform mat4 u_mvp_matrix;
uniform mat4 u_mv_matrix;
uniform mat3 u_normal_matrix;
uniform vec2 u_rot_center;
uniform float u_rot_angle;

varying vec3 v_pos_eye;
varying vec3 v_normal;
varying vec3 v_tangent;
varying vec2 v_texcoord;

vec2 rotate_uv(vec2 uv, vec2 center, float angle) {
    vec2 d = uv - center;
    float c = cos(angle);
    float s = sin(angle);
    return vec2(d.x * c - d.y * s, d.x * s + d.y * c) + center;
}

void main() {
    vec4 pos_eye = u_mv_matrix * vec4(a_vertex, 1.0);
    v_pos_eye = pos_eye.xyz;
    v_normal = u_normal_matrix * a_normal;
    v_tangent = u_normal_matrix * a_tangent;
    v_texcoord = rotate_uv(a_texcoord, u_rot_center, u_rot_angle);
    gl_Position = u_mvp_matrix * vec4(a_vertex, 1.0);
}
```

5.2 Fragment Shader (shader.gpu)

```
precision highp float;

uniform vec3 u_light_pos;
uniform sampler2D u_tex_diffuse;
uniform sampler2D u_tex_specular;
uniform sampler2D u_tex_normal;
```

```

varying vec3 v_pos_eye;
varying vec3 v_normal;
varying vec3 v_tangent;
varying vec2 v_texcoord;

void main() {
    vec3 diffuse_color = texture2D(u_tex_diffuse, v_texcoord).rgb;
    float specular_intensity = 1.0 - texture2D(u_tex_specular, v_texcoord).r;
    vec3 normal_sample = texture2D(u_tex_normal, v_texcoord).rgb;
    vec3 tangent_normal = normal_sample * 2.0 - 1.0;

    vec3 N = normalize(v_normal);
    vec3 T = normalize(v_tangent - dot(v_tangent, N) * N);
    vec3 B = cross(N, T);
    mat3 TBN = mat3(T, B, N);
    vec3 mapped_normal = normalize(TBN * tangent_normal);

    vec3 L = normalize(u_light_pos - v_pos_eye);
    vec3 V = normalize(-v_pos_eye);
    vec3 R = reflect(-L, mapped_normal);

    float ambient = 0.12;
    float diff = max(dot(mapped_normal, L), 0.0);
    float spec = pow(max(dot(R, V), 0.0), 32.0);

    vec3 final_color = ambient * diffuse_color
        + diff * 0.7 * diffuse_color
        + spec * 0.6 * vec3(specular_intensity);

    gl_FragColor = vec4(final_color, 1.0);
}

```

5.3 Center Point Control (main.js)

```

function handle_key(e) {
    const step = 0.025;
    const key = e.key.toLowerCase();

    switch (key) {
        case 'a':
            rot_center_u = Math.max(0, rot_center_u - step);
            break;
        case 'd':
            rot_center_u = Math.min(1, rot_center_u + step);
            break;
    }
}

```

```

        case 'w':
            rot_center_v = Math.min(1, rot_center_v + step);
            break;
        case 's':
            rot_center_v = Math.max(0, rot_center_v - step);
            break;
        default:
            return;
    }
    update_center_display();
}

document.addEventListener('keydown', handle_key);

```

5.4 Surface Parametric Function (Model.js)

```

const R1 = 0.5;
const R2 = 1.5;
const B = 1.0;

function calc_radius(alpha) {
    const k = Math.PI / (2 * B);
    return (R2 - R1) / 2 * (1 - Math.cos(k * alpha)) + R1;
}

function surface_point(alpha, beta) {
    const r = calc_radius(alpha);
    return [
        r * Math.cos(beta),
        r * Math.sin(beta),
        alpha - B
    ];
}

```